Guia del flux de treball amb Git

Bloc Transversal de PE (FIB-UPC) · Guia per a principiants · Repositori `Repo_PE_216`

1. Què és Git i què és GitHub?

Git és un programa que guarda l'**historial** de tots els canvis del projecte. Cada vegada que guardes una "foto" del projecte fas un **commit**. Si alguna cosa es trenca, sempre pots tornar a una foto anterior.

GitHub és la web on hi ha la **còpia compartida** del projecte. Cadascú té una còpia al seu ordinador, i GitHub fa de punt de trobada:

- **pull** = baixar de GitHub els canvis dels companys.
- **push** = pujar a GitHub els teus canvis.

2. Podem treballar tots tres a `main` sense trepitjar-nos?

> **Sí.** Git **no sobreescriu mai** la feina d'un company. Quan baixes canvis, Git els **fusiona** automàticament amb els teus:

Situació	Què passa
Treballeu en **fitxers diferents**	✅ Es fusiona sol. Cap problema.
Treballeu en el **mateix fitxer** però en **parts diferents**	✅ Es fusiona sol. Cap problema.
Dues persones canvien **les mateixes línies**	⚠️ **Conflicte**: Git s'atura i us pregunta quina versió voleu. No es perd res (vegeu l'apartat 6).

Per això és molt recomanable **repartir-se els fitxers**: per exemple, una persona fa el codi C++, una altra l'script d'R i una altra redacta el protocol.

3. Configuració inicial (només la primera vegada)

3.1 Baixar el repositori

```
git clone https://github.com/martinmol2007/Repo_PE_216.git
cd Repo_PE_216
```

El propietari del repositori us ha d'afegir com a **col·laboradors** a GitHub (*Settings* → *Collaborators*). Si no, no podreu fer push.

3.2 Dir a Git qui ets

```
make config NOM="Nom Cognom" EMAIL="el_teu_correu@exemple.com"
```

Així cada commit portarà el vostre nom i sabrem qui ha fet cada canvi (útil per a l'annex de contribucions de l'informe).

4. El cicle de treball de cada dia

1. make pull
Baixa la feina dels companys
→
2. Treballa
Edita el codi, el protocol...
→
3. make
Comprova que compila
→
4. make save
Guarda i puja a GitHub

```
# 1. Abans de començar, SEMPRE:
make pull

# 2. Treballa normalment amb el teu editor...

# 3. Comprova que el codi compila:
make

# 4. Quan acabis (o cada cop que facis una part important):
make save MSG="Afegit l'algorisme naive"
```

> **Les dues regles d'or:**
> 1. Fes `make pull` **abans** de començar a treballar.
> 2. Fes `make save` **quan acabis**. No deixis canvis sense pujar durant dies: com més temps passa, més probable és que hi hagi conflictes.

Què fa exactament `make save`?

1. **Compila** tots els `.cc`. Si no compila, s'atura i **no puja res** (així no trenquem el codi als companys).
2. **Commit**: guarda tots els canvis amb el missatge de `MSG`.
3. **Pull**: baixa i fusiona la feina que hagin pujat els companys mentrestant.
4. **Push**: puja-ho tot a GitHub.

El missatge del commit

Al començament del `Makefile` hi ha la variable `MSG`. Podeu canviar-la allà, o bé (millor) escriure el missatge directament a l'ordre:

```
make save MSG="Protocol: escrita la seccio de variables"
```

Un bon missatge explica **què** heu fet en una frase curta. Eviteu missatges com "canvis" o "asdf".

Coautors: quan feu una part entre dues o tres persones

Si heu fet una part junts (per exemple, dos membres programant al mateix ordinador), només un fa el commit, però podeu posar els altres com a **coautors**. A GitHub el commit sortirà amb tots els noms, i queda registrat per a l'annex de contribucions.

Primer (un sol cop): al començament del `Makefile`, poseu el nom i el correu de GitHub de cada membre:

```
MEMBRE_1 = Nom Cognom <correu_de_github@exemple.com>
MEMBRE_2 = ...
MEMBRE_3 = ...
```

Després, afegiu `AMB=` amb el número dels membres que han col·laborat:

```
# Amb el membre 2 com a coautor:
make save MSG="Escrit l'apartat de variables" AMB=2

# Amb els membres 2 i 3:
make save MSG="Analisi en R" AMB="2 3"

# Amb una persona que no és a la llista:
make save MSG="..." COAUTOR="Nom Cognom <correu@exemple.com>"
```

> Perquè GitHub reconegui el coautor, el correu ha de ser **el mateix que té al seu compte de GitHub**.

5. Totes les ordres del Makefile

Ordre	Què fa
`make`	Compila tots els fitxers `.cc` de l'arrel. Els executables van a la carpeta `bin/`.
`make run P=test`	Compila i executa el programa `test` (posa el nom del `.cc` sense l'extensió).
`make clean`	Esborra els executables.
`make config NOM="..." EMAIL="..."`	Configura el teu nom i correu (primera vegada).
`make status`	Mostra quins fitxers has canviat.
`make pull`	Baixa els canvis dels companys.
`make commit`	Només guarda els canvis en local (sense pujar-los).
`make push`	Només puja els commits a GitHub.
`make save`	**Tot en un**: compila + commit + pull + push. És l'ordre que fareu servir gairebé sempre.
`make save MSG="..." AMB=2`	Com `make save`, però posant el membre 2 com a coautor (`AMB="2 3"` per a diversos).
`make log`	Mostra els últims commits (qui ha fet què).
`make help`	Mostra aquesta llista al terminal.

6. Què és un conflicte i com es resol

Si tu i un company heu modificat **les mateixes línies** d'un fitxer, `make pull` (o `make save`) s'aturarà amb el missatge **CONFLICTE**. Git escriurà dins del fitxer les dues versions, així:

```
<<<<<<< (versio de GitHub, la del company)
cout << "Introdueix n: ";
=======
cout << "Escriu un nombre: ";
>>>>>>> (la teva versio)
```

Passos per resoldre'l

1. Executa `git status` i mira quins fitxers surten com a **both modified**.
2. Obre cada fitxer i busca les marques `<<<<<<<`. (VS Code les marca en colors i té botons com *Accept Current / Accept Incoming / Accept Both*.)
3. Deixa el codi com ha de quedar i **esborra les tres línies de marques**.
4. Guarda el fitxer i executa:

```
git add nom_del_fitxer
git rebase --continue
make push
```

> **Si et perds:** `git rebase --abort` deixa tot exactament com estava abans del pull. Després demana ajuda al grup.

7. Errors habituals

Missatge / situació	Solució
`rejected ... (fetch first)` o "GitHub ha rebutjat el push"	Un company ha pujat canvis abans que tu. Fes `make pull` i després `make push` (o directament `make save`).
`Please tell me who you are`	Fes `make config NOM="..." EMAIL="..."`.
`Permission denied` / `403` en fer push	No ets col·laborador del repositori, o cal iniciar sessió a GitHub (amb un *token* com a contrasenya).
"No hi ha canvis per guardar"	No has modificat res des de l'últim commit. No passa res.
Error de compilació en fer `make save`	Arregla l'error primer. El Makefile no puja codi que no compila.
Has treballat sense fer `make pull` abans	Normalment no passa res: `make save` ja fa el pull abans del push. Com a molt hi haurà un conflicte (apartat 6).

8. Bones pràctiques per al grup

- **Repartiu-vos els fitxers** i avisa pel xat del grup si has de tocar un fitxer d'un altre.
- Feu **commits petits i freqüents** amb missatges clars.
- **No pugeu** executables ni fitxers temporals: el `.gitignore` ja els exclou automàticament.
- Els documents Word (`.docx`) no es poden fusionar línia a línia: **que només una persona l'editi alhora**, o feu servir Google Docs per redactar i pugeu-ne la versió final.
- Les dades (`.csv`) i els scripts d'R **sí** que s'han de pujar: formen part del lliurament.

9. Glossari

Paraula	Significat
Repositori (repo)	La carpeta del projecte amb tot el seu historial.
Commit	Una "foto" guardada del projecte, amb un missatge i un autor.
main	La branca principal: la línia de treball compartida per tots.
Pull	Baixar i fusionar els canvis de GitHub.
Push	Pujar els teus commits a GitHub.
Conflicte	Dues persones han canviat les mateixes línies; cal triar a mà quina versió es queda.
Coautor	Persona que ha col·laborat en un commit que ha fet un altre membre.
.gitignore	Llista de fitxers que Git no ha de pujar mai.
Makefile	Fitxer amb ordres preparades (`make save`, `make pull`...) per no haver d'escriure ordres de Git llargues.